

Employee Payroll & Attendance Management System

A Console-Based Java Application for Employee Records, Attendance Tracking, and Automated Payroll Generation

Project Report

Course: Programming in Java

Submission Type: Build Your Own Project (VITyarthi)

Technology Stack: Java 21 (JDK Standard Library)

1. Introduction

Payroll mistakes are expensive, and attendance records kept on paper or in scattered spreadsheets make those mistakes easy to make. Organizations need a reliable way to tie two things together correctly every month: who actually showed up for work, and how much each person should be paid as a result. This project, the **Employee Payroll & Attendance Management System**, is a console-based Java application that solves this by giving HR staff and employees a single consistent tool for the full monthly HR cycle: registering employees, marking daily attendance, and generating payslips whose deductions are always derived directly from the attendance records on file — never entered by hand.

The project was built to demonstrate core Java concepts covered in the course: object-oriented design (abstraction, inheritance, polymorphism), the Collections framework, custom checked exceptions, file I/O, and defensive, modular programming practice.

2. Problem Statement

Small and mid-sized organizations frequently rely on paper registers or disconnected spreadsheets to track who came to work and how much each person should be paid. This creates three recurring problems: attendance gets recorded inconsistently or not at all, payroll calculations are done by hand and prone to error, and there is no single place to check an employee's attendance and pay history together. The Employee Payroll & Attendance Management System addresses this by giving HR staff and employees a single, consistent application to manage registration, daily attendance, and monthly payroll generation, with the pay calculation always derived directly from the attendance records actually on file.

Target users: an HR Admin who registers employees, marks attendance, and generates payroll, and Employees who log in to check their own attendance and pay history without needing to go through HR for every query.

3. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output structure and a logical user workflow:

Module 1 — User Management

Registration and login for two roles, HR Admin and Employee. Passwords are hashed (never stored in plain text) and the role determines which menu is shown after login.

Module 2 — Attendance Management

HR Admin marks one attendance entry per employee per day (Present / Absent / Leave / Half-day). Duplicate entries for the same employee on the same day are rejected outright, so the attendance calendar can never be double-counted.

Module 3 — Payroll Processing

HR Admin generates a monthly payslip per employee: the employee's base salary is pro-rated against the days actually marked present that month, with an automatic deduction for each absent day, and the system refuses to generate payroll twice for the same employee/month. Employees can view their own attendance history and payslip history at any time.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Employees are looked up by employee ID/username via a HashMap-backed lookup, giving $O(1)$ average access instead of linear scans across all users.
Security	Passwords are hashed with SHA-256 (<code>java.security.MessageDigest</code>) before being written to disk; plain-text passwords are never persisted or logged.
Reliability	All employee, attendance, and payroll data is written to disk after every mutating operation, so state survives an application restart or crash.
Usability	A simple, numbered, menu-driven console interface guides both HR Admin and Employee through every available action.
Maintainability	Code is split into model / service / util / exception packages, each with a single clear responsibility, making changes localized and low-risk.
Error Handling	Custom checked exceptions (<code>EmployeeNotFoundException</code> , <code>AttendanceAlreadyMarkedException</code> , <code>PayrollAlreadyGeneratedException</code> , <code>InvalidCredentialsException</code> , <code>DuplicateUserException</code>) make failure cases explicit and force calling code to handle them.
Logging / Monitoring	Every significant action (login, employee registered, attendance marked, payroll generated, errors) is timestamped and appended to <code>logs/app.log</code> .
Resource Efficiency	The application has no external runtime dependencies and keeps its full working set in memory, backed by lightweight flat-file persistence.

5. System Architecture

The application follows a simple layered architecture. The presentation layer (`Main.java`) only handles console input/output and delegates all decisions to the service layer. The service layer (`AuthService`, `AttendanceService`, `PayrollService`) contains all business rules; notably, `PayrollService` reads monthly attendance summaries from `AttendanceService` rather than duplicating that logic, keeping attendance as the single source of truth for pay calculations. The model layer holds the domain objects (User hierarchy, `AttendanceRecord`, `PayrollRecord`). The utility layer provides cross-cutting concerns — file persistence, password hashing, and logging — that all three services depend on.

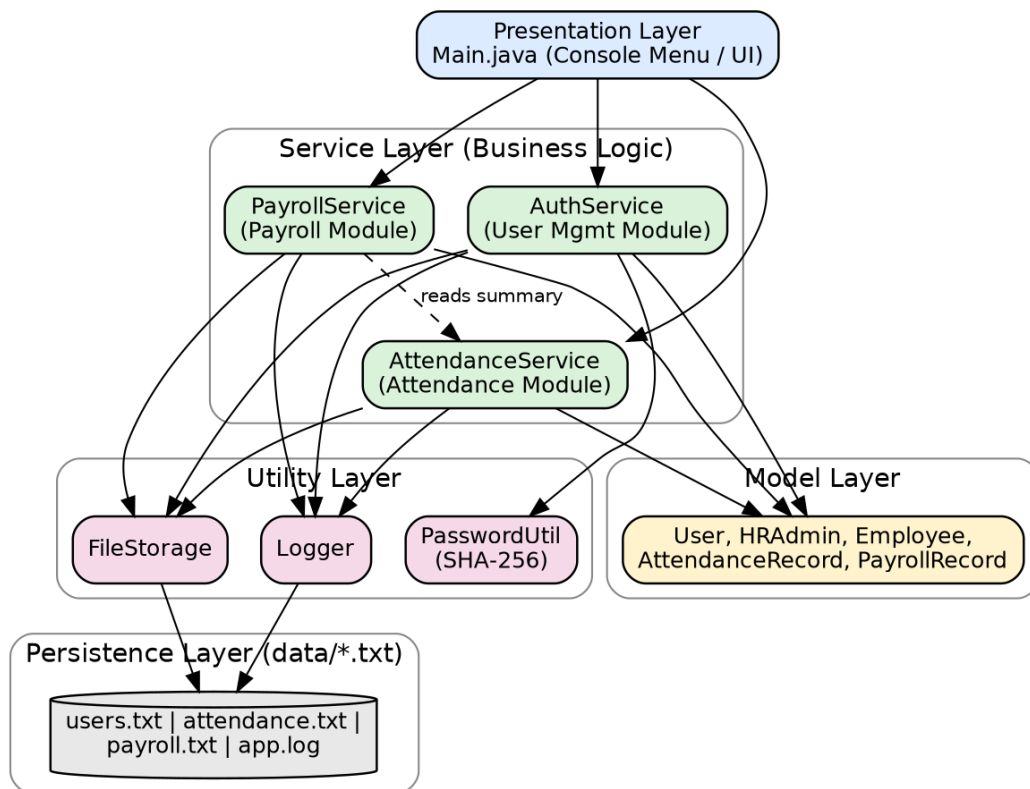


Figure 5.1: Layered system architecture.

6. Design Diagrams

6.1 Use Case Diagram

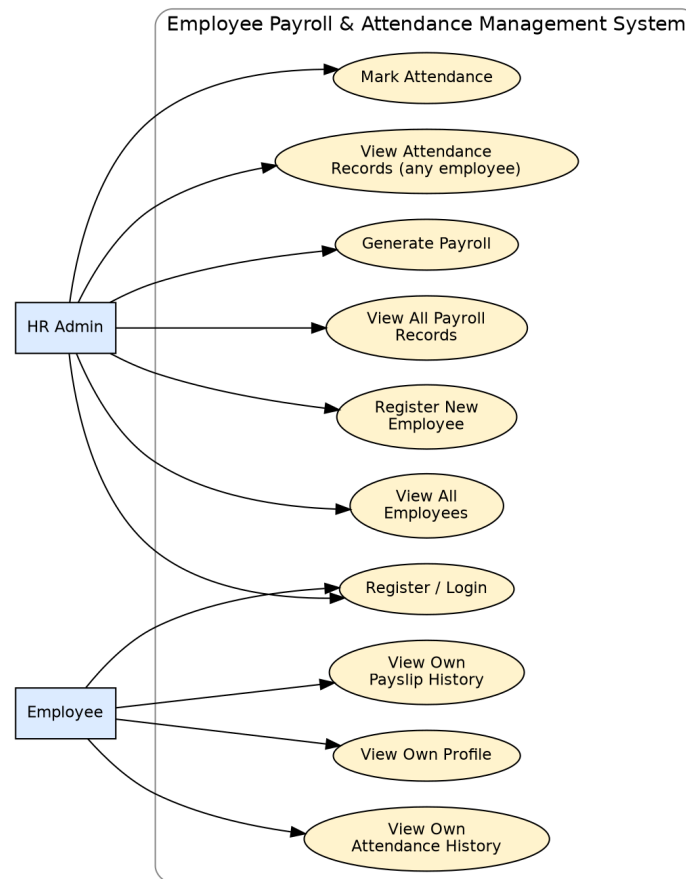


Figure 6.1: Use case diagram for HR Admin and Employee actors.

6.2 Process / Workflow Diagram

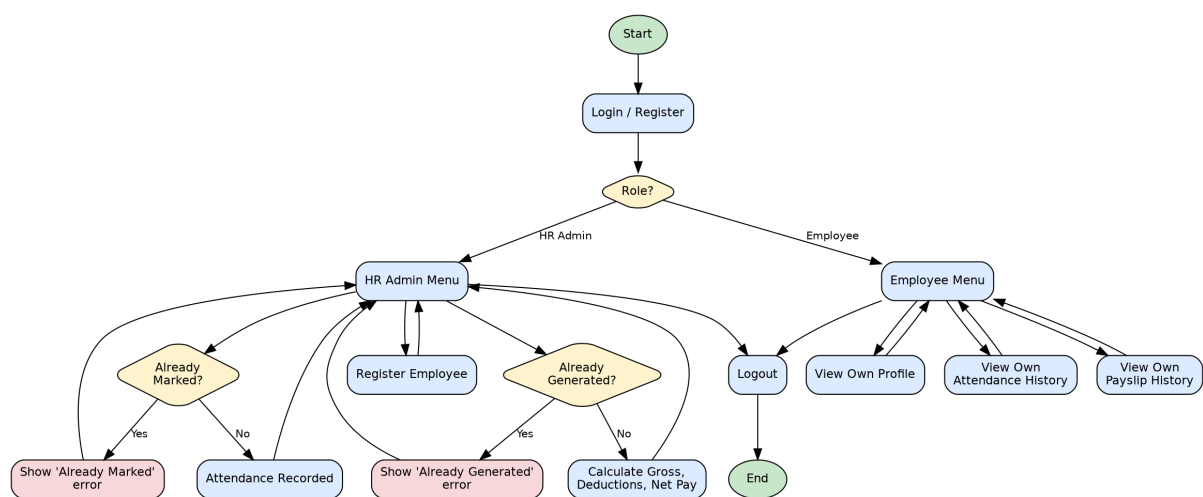


Figure 6.2: End-to-end user workflow from login to logout.

6.3 Sequence Diagram — Generate Payroll

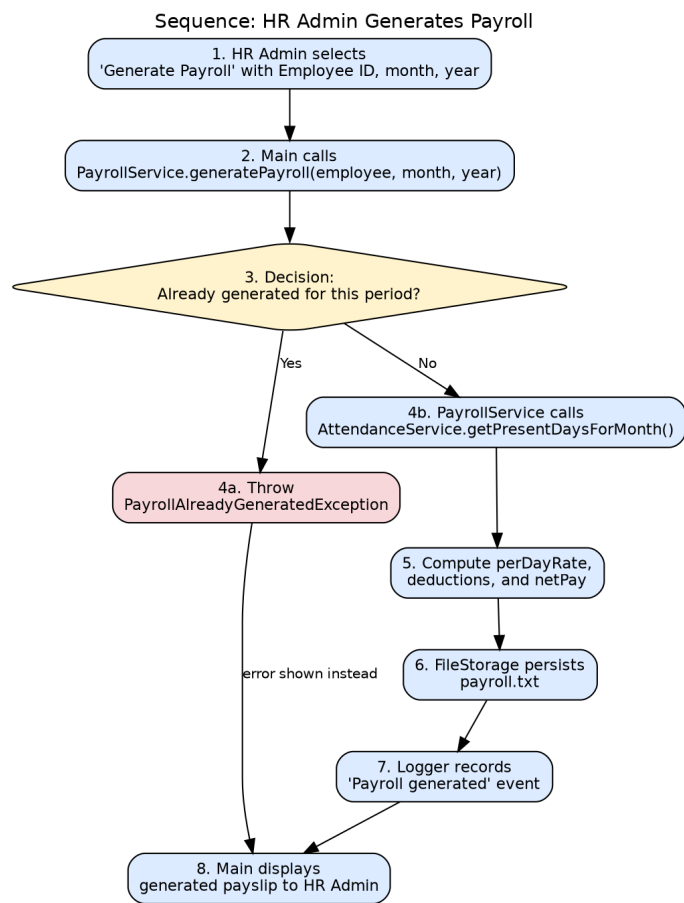


Figure 6.3: Sequence of calls when HR Admin generates a payslip.

6.4 Class Diagram

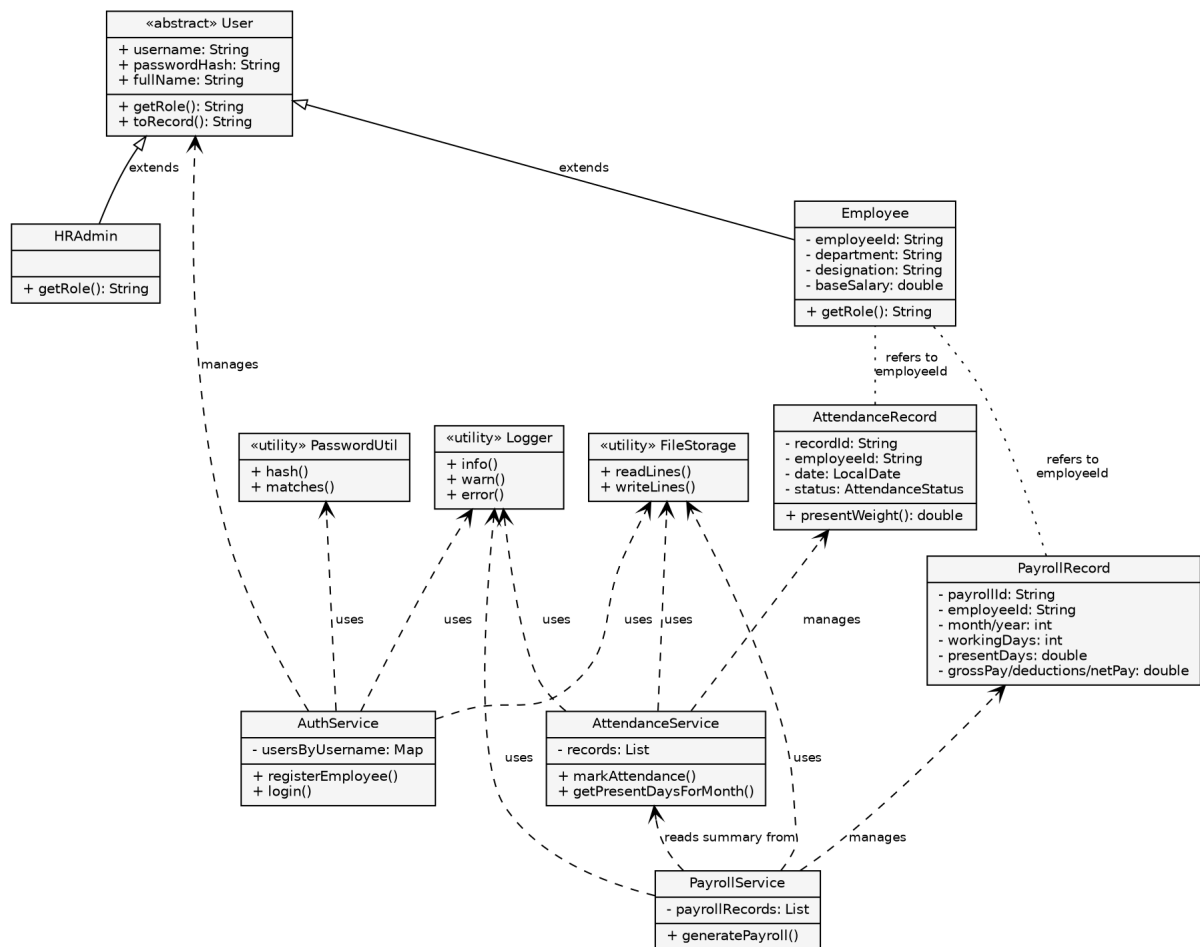


Figure 6.4: Core class relationships (inheritance and usage).

6.5 Data / Storage Schema

The project uses flat-file persistence rather than a relational database, so an ER diagram is replaced with an equivalent file-schema diagram showing the same key relationships (**attendance.txt** and **payroll.txt** both reference **users.txt** by employee ID, and payroll aggregates that month's attendance).

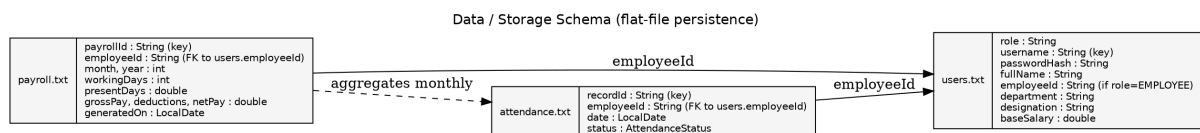


Figure 6.5: Data schema across the three persisted files.

7. Design Decisions & Rationale

Flat-file persistence instead of a database. Keeps the project dependency-free and runnable anywhere the JDK is installed, which matches the scope of a standalone course project. The `FileStorage` class isolates this choice, so swapping in JDBC/SQLite later would only touch one class.

Abstract User base class with HRAdmin/Employee subclasses. HRAdmin and Employee share almost all login state (username, password hash, full name) but Employee additionally carries HR-specific attributes (employee ID, department, designation, base salary). Modeling this with inheritance lets the rest of the code treat both polymorphically for login while still allowing Employee to carry extra fields only it needs.

Payroll always derives from attendance, never entered manually. `PayrollService` deliberately has no way to set a deduction directly — it can only ask `AttendanceService` for that month's present-day count and compute the deduction from it. This removes an entire class of data-entry error where payroll and attendance quietly disagree.

Custom checked exceptions over generic RuntimeExceptions. `AttendanceAlreadyMarkedException` and `PayrollAlreadyGeneratedException` in particular make two easy-to-make mistakes — double-marking a day, double-paying a month — impossible to do silently. Every failure path is explicit in the method signature, so the console layer is forced to handle it gracefully instead of crashing or, worse, corrupting payroll data.

HashMap for users, List for attendance/payroll records. Users are looked up by a unique key far more often than they are iterated in order, so HashMap gives $O(1)$ average access. Attendance and payroll records are mostly appended and scanned chronologically or by employee, which fits a List naturally.

SHA-256 password hashing. A one-way hash means the stored `users.txt` file never contains a recoverable password, even though the project does not implement network security — this keeps the security requirement meaningful without needing a database or server.

8. Implementation Details

The project has 16 source files organized into four packages under `com.payroll`, comfortably exceeding the minimum of 5–10 meaningful modules/classes/files expected for a coding project:

Package	Files	Responsibility
<code>com.payroll</code>	<code>Main.java</code>	Console UI / application entry point
<code>com.payroll.model</code>	<code>User</code> , <code>HRAdmin</code> , <code>Employee</code> , <code>AttendanceStatus</code> , <code>AttendanceRecord</code> , <code>PayrollRecord</code>	Domain objects
<code>com.payroll.service</code>	<code>AuthService</code> , <code>AttendanceService</code> , <code>PayrollService</code>	Business logic for all 3 modules
<code>com.payroll.util</code>	<code>FileStorage</code> , <code>Logger</code> , <code>PasswordUtil</code>	Persistence, logging, hashing
<code>com.payroll.exception</code>	5 custom exception classes	Explicit, typed error handling

Key Java concepts applied: abstract classes and inheritance (User hierarchy), enums (`AttendanceStatus`), interfaces implicitly via `Serializable`, the Collections framework (`HashMap`, `ArrayList`, streams for duplicate-checking), custom checked exceptions, try-with-resources for file I/O, `java.time.LocalDate` for attendance and payroll period arithmetic, and `java.security.MessageDigest` for password hashing.

9. Screenshots / Results

As this is a console application, the result below is a captured transcript of an actual run: an HR Admin logs in, registers an employee, marks three days of attendance (two present, one absent), views the records, and generates that employee's payroll for the month — correctly deducting one day's pay for the one absence. Log lines are printed live to the console (and also written to `logs/app.log`).

```
[2026-09-16 17:32:42] INFO   Loaded 0 user account(s) from disk.
[2026-09-16 17:32:42] INFO   No users found. Seeded default HR admin (username: hradmin / password:
hr12345).
[2026-09-16 17:32:42] INFO   Loaded 0 attendance record(s) from disk.
[2026-09-16 17:32:42] INFO   Loaded 0 payroll record(s) from disk.
[2026-09-16 17:32:42] INFO   === Employee Payroll & Attendance Management System starting up ===
=====
EMPLOYEE PAYROLL & ATTENDANCE MANAGEMENT SYSTEM
=====
Default HR Admin login -> username: hradmin | password: hr12345

===== WELCOME =====
1. Login
2. Register as Employee
3. Exit
Choose an option: Username: Password: [2026-09-16 17:32:42] INFO   User logged in: hradmin (HRADMIN)

Login successful. Welcome, Default HR Administrator (HRADMIN)

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Choose a username: Choose a password: Full name: Department: Designation: Base monthly
salary: [2026-09-16 17:32:42] INFO   Registered new Employee account: jdoe (E0001)
Registration successful! Your Employee ID is E0001. You can now log in.

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: [E0001] Jane Doe                | Engineering      | Software Engineer | Base Salary:
60000.00

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Enter Employee ID: Enter date (YYYY-MM-DD), or leave blank for today: Status: 1=PRESENT
2=ABSENT 3=LEAVE 4=HALF_DAY
[2026-09-16 17:32:42] INFO   Attendance marked: E0001 -> PRESENT on 2026-09-01
Attendance marked: A00001 | Employee:E0001 | Date:2026-09-01 | Status:PRESENT

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
```

```

5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Enter Employee ID: Enter date (YYYY-MM-DD), or leave blank for today: Status: 1=PRESENT
2=ABSENT 3=LEAVE 4=HALF_DAY
[2026-09-16 17:32:42] INFO Attendance marked: E0001 -> PRESENT on 2026-09-02
Attendance marked: A00002 | Employee:E0001 | Date:2026-09-02 | Status:PRESENT

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Enter Employee ID: Enter date (YYYY-MM-DD), or leave blank for today: Status: 1=PRESENT
2=ABSENT 3=LEAVE 4=HALF_DAY
[2026-09-16 17:32:42] INFO Attendance marked: E0001 -> ABSENT on 2026-09-03
Attendance marked: A00003 | Employee:E0001 | Date:2026-09-03 | Status:ABSENT

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Enter Employee ID: A00001 | Employee:E0001 | Date:2026-09-01 | Status:PRESENT
A00002 | Employee:E0001 | Date:2026-09-02 | Status:PRESENT
A00003 | Employee:E0001 | Date:2026-09-03 | Status:ABSENT

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Enter Employee ID: Month (1-12): Year (e.g. 2026): [2026-09-16 17:32:42] INFO Payroll
generated: P00001 for E0001 (9/2026) -> Net Pay: 40000.0
Payroll generated successfully:
Payslip #P00001 | Employee:E0001 | 09/2026 | Working Days:3 | Present:2.0 | Gross:60000.00 |
Deductions:20000.00 | Net Pay:40000.00 | Generated:2026-09-16

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option: Payslip #P00001 | Employee:E0001 | 09/2026 | Working Days:3 | Present:2.0 |
Gross:60000.00 | Deductions:20000.00 | Net Pay:40000.00 | Generated:2026-09-16

----- HR ADMIN MENU (hradmin) -----
1. Register New Employee
2. View All Employees
3. Mark Attendance
4. View Attendance Records (by Employee)
5. Generate Payroll for an Employee
6. View All Payroll Records
7. Logout
Choose an option:
===== WELCOME =====
1. Login
2. Register as Employee
3. Exit
Choose an option: [2026-09-16 17:32:42] INFO === Application shut down cleanly ===
Goodbye!

```

10. Testing Approach

The project was tested manually end-to-end by driving the console application through every menu path for both roles, and by inspecting the resulting data files and log output after each action. The scenarios covered:

- HR Admin login with the seeded default account, and a deliberately wrong password to confirm `InvalidCredentialsException` is raised and handled cleanly.
- Registering an employee, then registering another with the same username to confirm `DuplicateUserException` is raised.
- Marking attendance for an employee on a given date, then marking it again for the same date to confirm `AttendanceAlreadyMarkedException` is thrown instead of silently overwriting the record.
- Generating payroll for an employee/month with a mix of present and absent days, and verifying the deduction equals $(\text{absent days} \div \text{working days}) \times \text{base salary}$.
- Attempting to generate payroll twice for the same employee/month to confirm `PayrollAlreadyGeneratedException` is thrown instead of double-paying.
- Restarting the application after data had been written, to confirm employees, attendance, and payroll records are correctly reloaded from disk (reliability).
- Logging in as the employee just created and confirming their self-service views show exactly the attendance and payroll HR generated for them.
- Inspecting `logs/app.log` after a full session to confirm every action is timestamped and recorded.

11. Challenges Faced

Deciding what "working days" means for a deduction calculation. Rather than hard-coding a fixed number of working days per month (which breaks for partial months or organizations with different calendars), `workingDays` is defined as the number of attendance entries HR has actually marked for that employee in that month. This means a month is only "complete" once every expected day has been marked, which matches how a real HR team would operate and avoids a whole class of off-by-a-few-days bugs.

Keeping payroll and attendance consistent without duplicating data. Early versions considered storing a monthly summary field on the Employee object itself, but that would let payroll and the underlying attendance records silently drift apart if one was edited later. Routing every payroll calculation through `AttendanceService.getPresentDaysForMonth()` instead means there is exactly one source of truth for attendance.

Modeling half-day attendance cleanly. A half-day needed to count as 0.5 of a present day for payroll purposes without introducing a separate numeric field alongside the status enum. `AttendanceRecord.presentWeight()` resolves this by mapping each `AttendanceStatus` value to its pay weight in one place, so the enum stays purely descriptive and the arithmetic stays in one method.

12. Learnings & Key Takeaways

Building this project reinforced how much safer a system becomes when a downstream calculation (payroll) is only ever derived from an upstream record (attendance) rather than being entered independently — it structurally rules out a whole category of "the numbers don't match" bugs. Using checked exceptions for expected failure cases (a duplicate attendance entry, a double-generated payslip) rather than crashing made the console flow noticeably more trustworthy for something as sensitive as payroll. It also became clear how enums plus a single small mapping method (`presentWeight()`) can express business rules like "a half-day counts as half pay" far more cleanly than scattering conditional logic across the codebase.

13. Future Enhancements

- Replace flat-file storage with a real relational database (e.g. SQLite via JDBC) for concurrent multi-user access.
- Add a graphical (JavaFX/Swing) or web front end in place of the console UI.
- Support tax-slab-based statutory deductions and configurable pay components (HRA, bonuses, reimbursements) instead of a single base salary.
- Add a leave-balance system so "Leave" attendance entries can be checked against an employee's remaining entitlement.
- Export generated payslips as PDF documents employees can download.
- Add JUnit automated tests for the service layer to complement the manual testing performed for this submission.

14. References

- Oracle. *Java Platform, Standard Edition 21 API Specification*. <https://docs.oracle.com/en/java/javase/21/docs/api/>
- Oracle. *The Java Tutorials — Collections, Exceptions, and File I/O*. <https://docs.oracle.com/javase/tutorial/>
- Course material: Programming in Java (syllabus concepts applied throughout this project — OOP, collections, exception handling, file I/O).